A Process Framework for Evaluating GenAI Adoption and Use in Software Engineering

Liang Yu, Emil Alégroth, Panagiota Chatzipetrou, Tony Gorschek

***Abstract*—Generative Artificial Intelligence (GenAI) is increasingly adopted in software development for faster ideations, code assistance, and automation, from early exploration to operational deployment. However, organizations are uncertain about how to evaluate product quality and quality-in-use when using GenAI across these stages. Standards, such as ISO/IEC 25059, distinguish between software product quality (e.g., usability) and quality-in-use (e.g., satisfaction). However, there is a lack of empirical studies on applicability, usefulness, and evaluation methods in industrial practice. These insights are important for the trend of growing GenAI adoption.**

In this study, we investigate GenAI adoption and use, in particular, how quality is evaluated for adoption decisions, monitoring, and assessment of product quality. We conducted 19 semi-structured interviews, supported by archival data analysis for triangulation (15 documents and 184 web pages), in two companies belonging to a large organization, to understand adoption steps, quality concerns, evaluation practices, and role responsibilities.

Our findings describe a three-phase adoption process – Ideation, Development, and Operation – highlighting where and how quality evaluations occur, the criteria used, and the distribution of responsibilities. We synthesize these insights into an evidence-based quality evaluation framework.

Through a GenAI for Software Engineering (AI4SE) use case, we conclude that our framework demonstrates its applicability in industrial practice for guiding structured quality evaluation. Furthermore, the results highlight a need for a coordinating role – the GenAI Quality Lead – to improve accountability and traceability in AI-based systems. This contributes to Software Engineering for AI (SE4AI) by clarifying responsibilities when developing GenAI-based systems.

***Index Terms*—Generative Artificial Intelligence, GenAI, AI4SE, SE4AI, Empirical Study, Software Engineering**

I. Introduction

Generative Artificial Intelligence (GenAI) [1] with Large Language Models (LLMs) [2] has been rapidly adopted in software engineering. This adoption covers two complementary perspectives: Software Engineering for AI (SE4AI), which provides practices for building reliable AI-enabled systems, and AI for Software Engineering (AI4SE), where GenAI assists traditional development tasks [3]. From requirements analysis to code generation [4], GenAI is transforming how software is developed and maintained [5]. Industrial GenAI adoption typically starts with prompt-based experimentation and iterative refinement, gradually maturing into production-ready implementations. This adoption introduces risks for software quality [6].

The main challenge is to define what to evaluate, when, and how — with adoption checkpoints and monitoring — for product quality and quality-in-use. Unlike traditional deterministic software [7], GenAI-based software produces probabilistic outputs that can vary across runs, even with identical inputs [6]. The generated content can be plausible but factually incorrect, biased, or potentially harmful, raising concerns around accuracy, security, and compliance [6], [8]. These characteristics – non-determinism, prompt and data sensitivity, and behavioral drift – limit the applicability of traditional software quality assurance techniques, which assume fixed logic and predictable outputs.

ISO/IEC 25059 extends the established ISO/IEC 25010 quality model to AI-enabled software [9], distinguishing between product quality (e.g., functional suitability, reliability, security) and quality-in-use (e.g., effectiveness, efficiency, freedom from risk) [10]. However, to the best of our knowledge, there is limited research on how these qualities are interpreted and evaluated when GenAI is used in industrial software development. In particular, it is unclear which quality aspects practitioners prioritise, when and how evaluations occur during the adoption, and who holds responsibility for them [11].

To address these questions, we investigated how engineers adopt GenAI, how quality evaluation is performed, and how responsibilities are distributed in two software development companies. We conducted a case study within two software development companies, following the guidelines set by Runeson et al [12]. A mixed-methods approach was used, combining interviews with analysis of internal web pages and documents, which enabled us to capture both documented processes and practices. Using the study findings, we designed an evaluation framework aligned with ISO/IEC 25059, and then validated the framework on a real-world GenAI use case.

Our contributions are:

- Empirical evidence on how GenAI is adopted and used in industrial software development.
- Insights on quality evaluation covering quality characteristics in ISO/IEC 25059, including evaluation aspects, metrics, and criteria used.
- A cross-functional role, the GenAI Quality Lead, was proposed to coordinate quality-related responsibilities across the GenAI adoption.

Liang Yu is with Dept. Software Engineering, Blekinge Institute of Technology, Karlskrona, Sweden. Email: liang.yu@bth.se

Emil Alégroth is with Dept. Software Engineering, Blekinge Institute of Technology, Karlskrona, Sweden. Email: emil.alegroth@bth.se

Panagiota Chatzipetrou is with Örebro University, Örebro, Sweden. Email: panagiota.chatzipetrou@oru.se

Tony Gorschek is with Blekinge Institute of Technology, Karlskrona, Sweden, and Fortiss, Munich, Germany. Email: tony.gorschek@bth.se

Corresponding author: Liang Yu.

- An evidence-based evaluation framework that aligns industry workflows with ISO/IEC 25059.
  - Validation on a real-world GenAI use case that shows applicability and decision support across adoption checkpoints.

Through investigating quality evaluation in industrial practices, our study helps organizations pursue responsible GenAI adoption and apply ISO standards in daily engineering work.

The remainder of this study is structured as follows. Section II presents background information and related work on GenAI in software engineering and quality evaluation approaches. Section III describes our research methodology, including research questions, case company details, and data collection and analysis methods. Section IV presents the proposed quality evaluation framework, and Section V illustrates the framework verification through a use case implementation. Section VI addresses threats to validity. Section VII discusses implications, including the potential emergence of new roles such as AI Quality Lead. Section VIII concludes the paper with a summary of contributions and future research directions.

II. Background and Related work

A. Background

GenAI models [2], such as GPT-4o, have been used in software engineering activities, including code generation [13], documentation [14], and requirements drafting [15]. In industry, the adoption of GenAI is not always tied to a complete "system" from the beginning [16]. It often begins with integrating a model into a development task or piloting an idea with prompts. This adoption view helps teams understand how quality considerations arise along GenAI adoption [11].

ISO/IEC 25059 [17] extends the ISO/IEC 25010 [18] quality model to AI-enabled software, that is, software integrating AI models or services whose outputs influence system runtime behaviors. It structures quality into two complementary categories:

- Product quality: characteristics include functional suitability, performance efficiency, reliability, security, maintainability, and portability [17].
- Quality-in-use: characteristics cover effectiveness, efficiency, satisfaction, freedom from risk, and context coverage [17].

Both categories are important to GenAI adoption and use. For example, 'functional suitability' concerns whether outputs satisfy specified functional requirements for the intended tasks, whereas 'freedom from risk' includes legal, ethical, and security considerations.

B. Related work

1) Quality evaluation methods for GenAI: Several methods for evaluating the quality of AI-based software exist. Unlike this work, Zhang et al. [3] introduced a structured quality framework addressing aspects such as data quality, model robustness, and integration maturity. Riccio et al. [9] developed testing strategies targeting correctness and validation challenges in AI-driven systems. Similarly, Breck et al. [19] proposed the ML Test Score, a checklist-based framework for assessing operational readiness and reducing hidden technical debt.

More recently, researchers have begun to explore evaluation techniques tailored to generative tasks such as code generation. Ribeiro et al. [20] proposed CheckList, a behavioral testing model for NLP systems that identifies failures in specific linguistic capabilities, such as negation handling and coreference resolution. Wang et al. [13] reviewed evaluation metrics for code generation, including functional correctness, code readability, and runtime performance. These approaches advance technical evaluation and can be applied in later development and product evaluation. Some of them offer guidance on system-level prototypes. However, their practical applicability, scalability, and suitability for industrial adoption remain uncertain due to limited validation in real-world contexts.

2) Empirical studies of GenAI in software engineering: Empirical research has examined how GenAI is applied in real-world software development contexts. Barke et al. [21] and Donvir et al. [11] investigated developer interactions with GenAI-assisted tools, reporting both productivity gains and new categories of errors introduced by model-generated code. Other studies have demonstrated GenAI's applicability in tasks such as test case generation [6], requirements analysis [22], and documentation [14].

These studies also raise concerns regarding quality and trust. Donvir et al. [11] observed that GenAI can produce subtle, hard-to-detect defects despite speeding up ideation and coding. Such defects are hard to detect because the generated code compiles and can pass limited tests while masking logic errors and missing checks. Non-determinism and prompt sensitivity for open-ended tasks make failures difficult to reproduce and verify. Aleti et al. [6] highlighted the difficulty of verifying correctness and security in GenAI-generated outputs. Park et al. [10] emphasised the importance of prompt engineering and developer awareness for achieving high-quality outputs.

3) Research gap: Existing literature focused on GenAI adoption does not provide a usage-oriented view of how quality is evaluated in industry. Our study addresses this gap by documenting the process of GenAI adoption in software development and mapping observed evaluation practices to ISO/IEC 25059.

III. Research methodology

We conducted a case study following Runeson et al.'s guidelines [12]. Archival data and semi-structured interviews were used to collect in-depth insights into how quality is evaluated throughout GenAI adoption in software development. An overview of the research process is shown in Fig. 1.

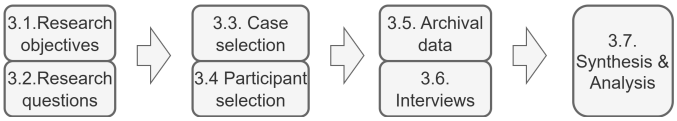

Fig. 1. Research process overview.

A. Research objectives

Our study objective is to investigate how industrial engineers adopt GenAI in software development. We focus on capturing the exact processes followed by practitioners. In parallel, we examine how software quality is evaluated throughout the use of GenAI to identify recurring practices or patterns. For example, the practices include which quality aspects are considered important, when and how evaluations are conducted, and who is responsible for carrying them out.

B. Research questions

To reach the research objectives, we formulate the following research questions:

- **RQ1:** What processes do industrial engineers follow when adopting GenAI in software development?
  This question breaks down adoption into phases and practical steps, covering model selection, system integration, and operational use. It provides the baseline for understanding the surrounding activities where quality concerns may emerge.
- **RQ2:** What quality aspects are evaluated across GenAI adoption and use in software development?
  This question focuses on the types of quality concerns that practitioners prioritize throughout the adoption and use, such as legal risk, security, or data handling.
- **RQ3:** Who is responsible for evaluating quality throughout GenAI adoption?
  Given the identified evaluation aspects, we examine which roles participate in quality-related work. It connects identified quality concerns to organizational ownership.
- **RQ4:** How is quality evaluation conducted throughout GenAI adoption?
  We investigate existing approaches to carry out the evaluations, which reveal how quality is judged in practice.

C. Case selection

We selected two software development companies, denoted as Company A and Company B, using purposive sampling [23] from our industrial network. Both belong to the same corporate group but operate in different domains (FinTech and Telecom). We anonymized the companies under non-disclosure agreements. When this study began, few external partners had established processes for using GenAI in software development. The selected companies offered ongoing AI initiatives and access to internal artifacts, enabling a focused in-depth case study. We discuss implications for generalizability in Section VI.

TABLE I
CONTEXT INFORMATION [24] OF THE SELECTED COMPANIES.

Context		Company A	Company B
Product	Business domain	FinTech	Telecom
	Product type	Finance services	Telecom support services
	Maturity	Mature product	Mature product
Organization	Number of employees	>500	>2000
	Corporate group	Same corporate group	Same corporate group
GenAI applications	number of AI initiatives	9	263

Table I presents the contexts of the selected companies. Company A produces a financial system (e.g., transactions, loans, and payments) that has successfully served customers for approximately two decades. Company B is in the telecommunications domain, providing support services for network management. Both companies operate globally and expand rapidly. Our data collection focused on the Swedish and Indian development sites. Both have mature software engineering practices and have set organizational goals to adopt GenAI in products. At the time of the study, 272 AI initiatives in total were recorded. Most of the recorded initiatives were ongoing, and some had been completed.

With many parallel AI initiatives, ad hoc practices lead to non-comparable results, duplicated effort, and limited visibility into risk and drift across teams. Shared adoption patterns and processes, such as common checkpoints (e.g., verification of data privacy), a minimal metric start-set (e.g., metrics on output validity/cost), clear role ownership (e.g., product owners), and traceable records (e.g., model versions and prompt templates), enable cross-team monitoring and incident review. This supports consistent decisions and reduces rework in legal and security reviews.

D. Participant selection

Participants for this study were selected using purposive sampling [23] following a multi-step process. Firstly, we identified active GenAI initiatives within each company using internal documentation (see Table I for initiative counts). Secondly, the first author was embedded with development teams in both companies, which provided us with sufficient understanding of the organization. Thirdly, this embedded access gave visibility into AI initiatives (teams, internal documentation, and demo sessions), which we used to identify participants across roles and responsibilities. We prioritized practitioners with hands-on GenAI responsibilities and direct involvement in quality evaluation activities. During interviews, we collected detailed information about participants' specific roles, geographic locations, and deeper insights into their GenAI experiences. As a result, shown in Table II, 19 participants were chosen.

TABLE II
PARTICIPANTS SELECTED FROM COMPANY A AND COMPANY B

ID	Participant role	NO. of participants	Working experience (years)	Company
1	Software developer	3	7 - 20	A (1), B (2)
2	Software architect	2	11 - 23	A (1), B (1)
3	Quality assurance engineer	3	8 - 19	A (1), B (2)
4	Legal engineer	3	9 - 23	A (1), B (2)
5	Security engineer	3	6 - 17	A (2), B (1)
6	Operations engineer	3	7 - 15	A (1), B (2)
7	Product owner	2	13 - 22	A (1), B (1)

E. Archival data

In addition to interviews, we collected archival data [12] (e.g., internal documents) to triangulate our findings and gain deeper insights into GenAI adoption and quality evaluation practices.

We examined a total of 184 internal wiki/web pages, which are counted as unique URLs, not A4 pages. Pages ranged from short information (1–2 paragraphs) to multi-section guides, and the count indicates breadth rather than physical length. These pages span a wide range of topics related to GenAI adoption, including GenAI development portals, legal compliance and risk assessment pages, GenAI experimentation guidelines, system development processes, and tooling documentation (e.g., API consoles and quality dashboards). Key categories included "GenAI risk and compliance", "AI model profiles", "development and assessment procedures", "GenAI initiative board" overviews, and "handbooks for GenAI practices". These resources provided important context on how quality evaluation is performed within organizations.

Furthermore, we analyzed 15 internal documents (e.g., PDF, Docx, Excel, slides), focused on how GenAI is adopted and evaluated in practice, such as "GenAI risk guardrails" (legal and technical risks), "AI adoption assessment" forms used at checkpoints, "Responsible GenAI" checklists, model registration and vendor assessment records, security and privacy reviews, metric definitions and dashboard specifications, and training slide decks.

These materials formed the primary evidence base, with concrete procedures, criteria, and role ownership. Firstly, they provided a high-level view of the GenAI adoption process, including handoffs and ownership that individual practitioners did not always see. Secondly, they enabled a cross-check through comparison with documented procedures and templates. Thirdly, they revealed organizational expectations for GenAI quality that were not yet consistently enacted in teams' daily work.

Although the archival data strengthened credibility, they also have limits. Access was restricted to internal platforms, so coverage reflects what was available during November 2024 to June 2025. As documents varied in detail and maintenance, we mitigated these risks by cross-checking with interviews.

F. Interviews

We conducted semi-structured interviews [12] to investigate how industrial practitioners adopt GenAI in software development and how they evaluate quality during this process. The interviews allowed for in-depth exploration of practices, decision rationales, and cross-role responsibilities not captured in internal pages, complementing the archival analysis.

A total of 19 interviews were held between November 2024 and June 2025. Twelve were in person and seven online, due to the geographical distribution of participants. The interviews were conducted by the first author and were held in English using an interview guide of 21 questions. The interview guide is available on the IEEEDataPort [25]. Each interview lasted between 46 and 60 minutes.

We took several actions to enhance interview validity. Firstly, we ensured role diversity in participant selection. Secondly, we refined the interview guide after pilot interviews based on emerging themes, then used the finalized guide consistently across all remaining interviews. Thirdly, we conducted member checking by sharing our analyzed data with participants for review through email and follow-up conversations to confirm our interpretations. We have obtained the consent information from all study participants.

However, certain limitations exist. As participants came from two companies in a large corporate group with ongoing AI initiatives, the sample may not reflect settings with minimal tooling, restricted data access, or without in-house legal and security support. Moreover, self-reporting can introduce recall bias. We used internal documents and web pages mentioned or shared by participants to confirm statements and elaborate on details from interview transcripts. However, we acknowledge that these sources may reflect similar organizational perspectives rather than providing fully independent triangulation.

G. Data synthesis and analysis

We employed thematic analysis [26] to identify patterns and themes in the collected data. This analysis followed a four-step process, supported by iterative collaboration and joint interpretation sessions.

1) *Initial coding*: A starting set of codes was generated from interview transcripts and archival documents. Codes were generated based on relevance to themes connecting to our research questions. For example, codes reflect GenAI adoptions (e.g., "prompting"), evaluation (e.g., "risk checks", "metrics"), and stakeholder roles (e.g., "product owner", "security").
2) *Refinement and disambiguation*: The initial codes were iteratively reviewed to improve granularity and avoid semantic overlap. For example, "correctness" was split into "output accuracy" and "code validity" to capture domain-specific distinctions.
3) *Theme construction*: We organized codes along two categories: process area (adoption phase and step) and quality dimension (ISO/IEC 25059 characteristics). We formed themes when repeated evidence appeared in both categories. Edge cases were resolved through team discussion to avoid overlap.
4) *Mapping to research questions*: Themes were organized according to their relevance to each research question. For example, codes concerning risk evaluations were connected to RQ2, while those related to developer responsibilities were connected to RQ3.

To support internal validity, coding was reviewed by the first three authors and reconciled through multiple discussion rounds. A shared coding protocol was used to align definitions, coding rationale, and theme development. Preliminary interpretations were verified through feedback sessions with selected participants, ensuring alignment with their experiences.

1) Deriving GenAI adoption and use processes: We present a four-step method to derive GenAI adoption processes based on the collected data. Firstly, *prompt piloting* in Ideation was derived from repeated interview statements about "quick prompt trials before building" and the Ideation step in Fig. 2, together with internal guidance on experimentation. Secondly, *legal and security risks* in Ideation were grounded in the risk checklist in Table III and interview accounts that approvals were required "before PoC," making it a checkpoint rather than

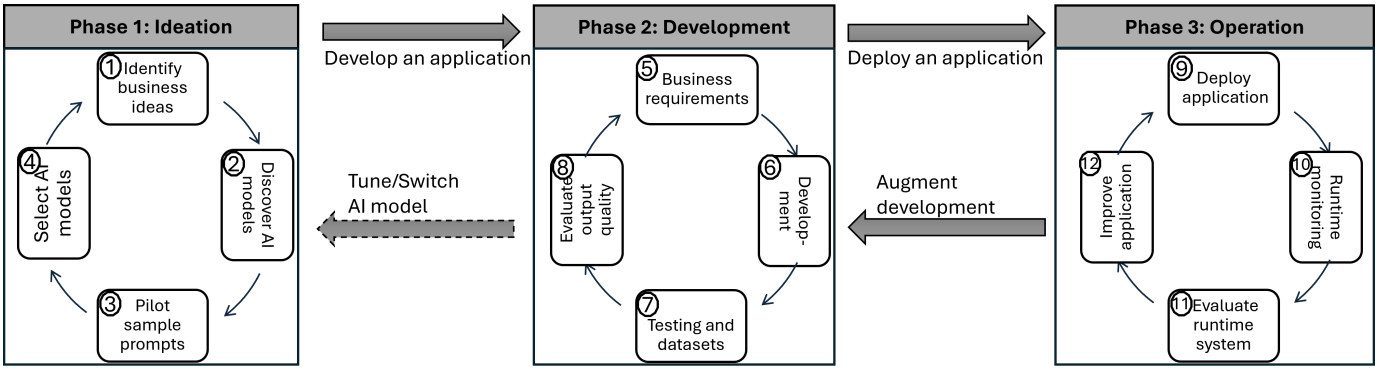

Fig. 2. Phases and steps followed to adopt GenAI.

an ad hoc review. Thirdly, *model selection* followed piloting and bridged into Development, as teams chose a provider on output suitability and integration ease, and then proceeded to *system integration* (Development step in Fig. 2). Fourthly, *runtime monitoring* in Operation was supported by the verification case evidence of dashboards and metrics (e.g., validity and latency) shown in Fig. 7, which then fed *continuous improvements*. We grouped labels such as "prompt tries," and "sandbox runs" into *prompt piloting*, and merged "legal review," "supplier terms," and "risk guardrails" into the *legal and security* category. For traceability, each step in the combined process was linked to its supporting interviews and documents, with an overview available on IEEEDataPoint [25].

IV. Results

We present study findings to answer our research questions in this section.

A. Results of RQ1 – What processes do industrial engineers follow when adopting GenAI in software development?

Fig. 2 shows the identified processes in phases for GenAI adoption and use in software development. The phases and subsequent steps were derived from our coding procedure in Section III-G1. The process is organized into three main phases: *Ideation*, *Development*, and *Operation*. These phases reflect a combination of exploratory use, iterative design, and deployment-oriented practices.

1) Phase 1: Ideation: This phase is initiated by identifying high-level business opportunities (Step 1). The ideas are usually sourced by product teams, often without strong technical feasibility input at this point. This reflects a trend where business-driven interest precedes technical alignment.

Once candidate ideas are selected, engineers explore potential GenAI solutions by surveying available models (Step 2). At this stage, engineers screen for early fit and risk rather than formal accuracy. They check task fit (outputs match the intended task and domain), data and policy fit (inputs allowed under privacy and security rules), legal acceptability of license, basic output quality on a few representative prompts, and integration feasibility (API access, hosting, deployment path).

Piloting sample prompts (Step 3) serves as a lightweight feasibility check. It allows teams to assess whether GenAI output aligns with expectations before further investment. In the studied companies, this step was often used to discard ideas quickly, reflecting a low-cost experimentation practice.

If the output appears promising, teams proceed to select a specific AI model (Step 4). Selection is commonly based on empirical prompt testing, vendor trust, and ease of integration, rather than formal quality guarantees. This also implies that model selection is frequently revisited as the project progresses.

2) Phase 2: Development: In the development phase, the focus shifts from ideation to system integration. Teams refine business requirements (Step 5) to accommodate GenAI capabilities and constraints, often reducing traditional specification granularity. In contrast to conventional software, requirements are shaped around acceptable variation in model output, making this step inherently iterative.

The development step (Step 6) involves incorporating the selected GenAI model into an application or service. This may include prompt engineering, workflow orchestration, or API-level integration. Participants noted that development often advances without complete clarity on evaluation criteria, reinforcing the need for later adjustments.

Testing and dataset preparation (Step 7) varies considerably in formality. Some teams curate specific test cases to validate expected behaviors, while others rely on production-like data to simulate real-world input. The level of rigor here depends on the perceived risk and target user.

Output evaluation (Step 8) is where the prototype is reviewed for its usefulness, correctness, and consistency. Unlike deterministic software, evaluation here involves human-in-the-loop judgment and often lacks predefined acceptance thresholds. This introduces challenges in comparing iterations or tracking progress over time.

3) Phase 3: Operation: After deployment (Step 9), systems enter the operation phase, where GenAI becomes part of production environments. This transition often marks a shift in responsibility from development teams to maintenance and operations teams.

Runtime monitoring (Step 10) is implemented to track anomalies, drift, or failure cases. However, monitoring practices remain immature; few teams employ systematic mechanisms for runtime evaluation beyond basic usage statistics.

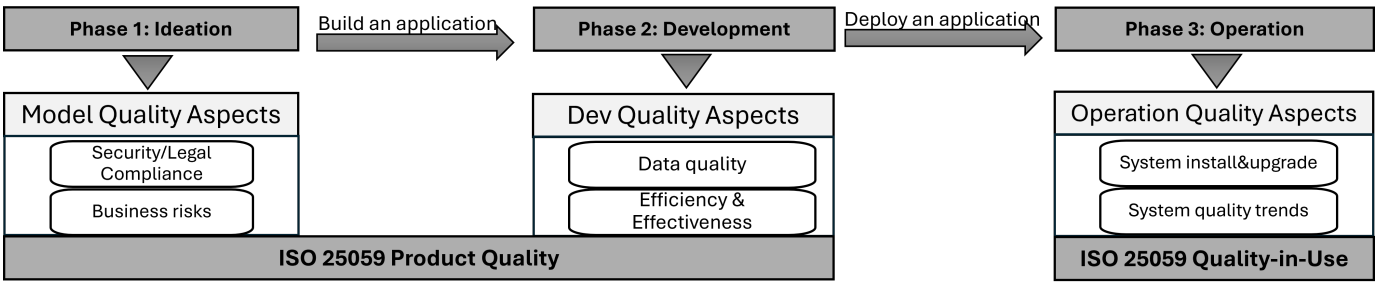

Fig. 3. Primary quality aspects considered throughout GenAI adoption and use, mapped to ISO/IEC 25059.

Evaluation of the runtime system (Step 11) focuses on stability, user satisfaction, and reliability in production contexts. This step is typically reactive—issues are addressed after surfacing, rather than through predictive assessments.

Improvement cycles (Step 12) are driven by operational feedback and usage data. These iterations may involve switching models, re-engineering prompts, or redesigning workflows. However, the improvement process is rarely governed by formal change management, reflecting the still-exploratory nature of GenAI operations.

4) Observations: We observed that GenAI adoption in industry follows a typical pattern of iterative experimentation, model probing, and incremental discovery, similar to how organizations adopt other new technologies. This reflects a shift from deterministic system design toward empirical convergence, where systems evolve through usage feedback and feasibility trials.

Moreover, we learned that quality evaluation occurs in most phases, but practices are often informal and subjective (e.g., accepting outputs without predefined thresholds for accuracy, latency, or safety). Practices such as prompt piloting, runtime observation, and prototype judgment serve as implicit gates for decision-making. This raises concerns about traceability and accountability in GenAI-based systems.

For practitioners, the identified three phases can serve as a reference to position their own efforts. It also offers researchers an example of how GenAI adoption unfolded in two industrial settings.

B. Results of RQ2 – What quality aspects are evaluated across GenAI adoption and use in software development?

Fig. 3 summarizes the quality aspects identified by participants across the three phases of GenAI adoption and use. Quality aspects in Phases 1 and 2 connect to *Product Quality*, and Phase 3 is about operation, which belongs to *Quality-in-Use* characteristics according to ISO/IEC 25059. These aspects reflect where practitioners currently focus their attention, shaped by their roles and organizational constraints.

1) Model Quality Aspects (Phase 1) — ISO 25059 Product Quality:

a) Security and legal compliance: Participants highlighted the need to understand legal implications early, especially concerning intellectual property, data privacy, and third-party model licensing. This concern is not only about selecting 'safe' models but also about whether the use case is legally viable. The motivation stems from the uncertainty around AI model behavior and ownership of training data, which introduces legal risks. Practitioners noted that without early legal validation, entire business ideas may later be blocked or delayed.

b) Business risks: Feasibility was often framed in business rather than technical terms, for example, unclear value propositions, unpredictable behaviour in user-facing contexts, or prohibitive costs. Participants noted that even technically functional GenAI models might conflict with user trust or supportability goals. As a result, early-phase quality evaluation often prioritizes business viability alongside compliance.

c) Actions: At this phase, quality evaluation focuses on compliance and purpose. Organizations are advised to involve legal, compliance, and product strategy teams early to assess feasibility before prototyping.

2) Development Quality Aspects (Phase 2) — ISO 25059 Product Quality:

a) Data quality: Data quality emerged as a central concern. Participants were uncertain whether test datasets and prompts accurately reflected real-world adoption, especially when outputs were highly sensitive to subtle input changes. In some teams, overfitting prompts to hand-crafted cases reduced generalization after deployment.

b) Efficiency and Effectiveness: Effectiveness was interpreted contextually: some defined it as "acceptable output for task completion", others included time-to-answer or resource efficiency (e.g., latency, token cost). Formal benchmarks were rarely used, according to participants. Instead, teams used quick iterations with subjective judgment. While agile, this informality makes scaling and repeatability difficult.

c) Actions: Informal assessments should be complemented with measurable criteria, for example, output consistency, processing time, and task success rates. Data quality should be revisited during development, not assumed from earlier checks.

3) Operation quality aspects (Phase 3) — ISO 25059 Quality-in-Use:

a) System installability and upgradeability: Participants reported deployment friction for GenAI components due to version mismatches, hosting constraints, and unclear dependencies. Such issues are more disruptive for GenAI than traditional systems, given their dynamic dependencies and licensing restrictions.

b) System quality trends: Monitoring quality trends in production was valued but underdeveloped. Most participants

lacked formal systems in place to track model drift, unexpected failures, or changing behavior across environments. Instead, most relied on user complaints or ticket reports from support teams. This limits feedback loops and hinders root-cause analysis.

c) Actions: Practitioners should treat GenAI-based components as evolving artifacts. Continuous monitoring, logging, and sampling should be used to detect unexpected changes, supported by feedback channels from users and operations teams.

4) Observations: While the identified aspects align with ISO/IEC 25059, several characteristics remain under-addressed:

- **Functional adaptability** — system responsiveness to dynamic environments or evolving data.
- **Robustness** — stable performance under bias, adversarial inputs, or novel conditions.
- **Transparency** — clear documentation of AI models' logic, data provenance, and decision rationale.
- **Societal and ethical risk mitigation** — adherence to fairness, privacy, and accountability principles.

During the interviews, these quality characteristics were rarely mentioned unless directly prompted. For instance, *functional adaptability* – an important reason for choosing GenAI in the first place—was not systematically evaluated. *Robustness* and *transparency* were typically implicit or informal (e.g., "seems to work well") rather than captured through structured criteria or logging. Without explicit evaluation, systems may degrade in unobserved ways, particularly after deployment.

Engineering teams should integrate selected ISO/IEC 25059 attributes into lightweight quality evaluation checklists or design review templates. The following aspects are actionable:

- **Transparency:** Document prompt structures, data origins, and model versioning. Use structured logging for prompt-output behavior over time.
- **Robustness:** Evaluate and guard against structured-output (e.g., JSON/XML) failures. Use schema validators (e.g., JSON Schema, XSD), constrained decoding or typed function calling, and strict parsers. Add automatic repair, enforce timeouts, and use retries with backoff and circuit breakers. Define fallbacks for known failure cases (e.g., cached answers, rule-based generation, or human-in-the-loop).
- **Societal and ethical risk mitigation:** Align GenAI adoption with Responsible AI frameworks [27]. Include non-engineering stakeholders early, such as legal, security, compliance officers, domain experts, and representative users.

These steps do not require full ISO compliance but can raise quality awareness and connect daily GenAI practice with standardized guidance.

C. Results of RQ3 – Who is responsible for evaluating quality throughout GenAI adoption?

GenAI integration into software development creates new quality challenges, with responsibility for addressing these challenges remaining fragmented across roles. Fig. 4 summarizes the roles identified in each phase of GenAI adoption—ranging from business ideation to runtime operations—and the specific quality concerns associated with them.

1) Responsible roles: In the **Ideation** phase, responsibility is distributed among business owners, legal and licensing teams, and security units. Business owners assess use case viability and business risks, while security and legal experts validate compliance with regulations and intellectual property policies when selecting models. As a participant stated: "We had to get the legal team involved early because we weren't sure if the use case was even allowed with the data we had."

In the **Development** phase, developers and architects lead integration, while quality engineers and analysts support testing and validation. Yet evaluation practices often remain informal or rely on domain intuition. Several participants reported: "It is mostly trial and error—engineers test prompts, tweak outputs, and we go with what feels right."
Some teams assign evaluation duties to developers by default, and in the absence of structured criteria, they rely on domain expertise and human judgement. Several participants noted: "There is no fixed checklist. We just look at whether the response makes sense or is dangerous."

In the **Operation** phase, roles such as infrastructure engineers and support analysts become involved. However, runtime feedback mechanisms are often weakly connected to earlier evaluation decisions. Participants stated that "We rely on support tickets to learn when something's off. There's no automated alerting tied to model output yet."

2) The need for a dedicated role in ownership: Across all phases, we found limited unified ownership for GenAI quality evaluation. Although responsibilities are scattered among domain experts, the lack of coordination and traceability reduces the accountability and repeatability of evaluation activities. For example, a participant stated: "Everyone is CERTAIN that SOMEONE is checking the quality, but no one knows who actually does it." This ambiguity is not unique to GenAI. Conventional projects may also suffer from diffuse quality ownership in siloed organizations. GenAI magnifies the problem because quality decisions cut across legal, security, compliance, and software development.

A recurring theme in interviews was the absence of a designated role that integrates technical, legal, and ethical concerns across the GenAI adoption and use. Therefore, we propose the role of **GenAI quality lead**. This role is not merely a test engineer or auditor but a cross-functional *driver*. The driver runs quality checks, records decisions, and can block or escalate unresolved risks. The aim is to ensure the evaluability, explainability, and appropriateness of GenAI features across the identified phases. As participants repeated that "It would help to have someone who knows what to check, not just technically, but in terms of risk or impact too."
Responsibilities of the quality lead include:

- Facilitate early-phase risk identification (e.g., legal, security, bias).
- Establish and adapt quality criteria with legal, security, and domain stakeholders.

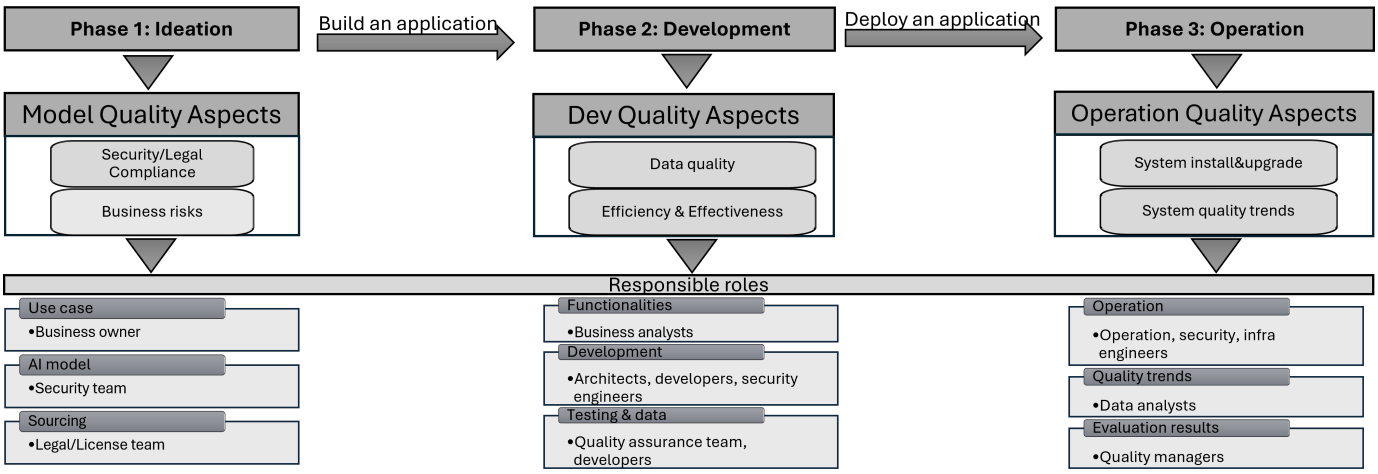

Fig. 4. Responsible roles for quality evaluation throughout GenAI adoption.

- Defining evaluation criteria with product and engineering teams.
- Drive evaluation practices across development and operations (e.g., output testing, runtime monitoring).
- Translating quality standards (e.g., ISO/IEC 25059) into project-specific practices.
- Serve as a contact point for review boards, audits, and compliance alignment.

This role builds upon principles of SE4AI, which emphasizes quality assurance throughout the AI engineering lifecycle. By formalizing such a role, companies can better integrate emerging standards (e.g., ISO/IEC 25059) and internal Responsible AI guidelines into everyday practice. Importantly, the role is not a replacement for domain-specific experts but a coordinating role that improves quality accountability and traceability. It helps move beyond ad hoc evaluations by embedding structured thinking into the day-to-day engineering of GenAI-based solutions.

D. Results of RQ4 – How is quality evaluation conducted throughout GenAI adoption?

Based on our findings from RQ1 to RQ3, we synthesized a process-oriented framework that integrates GenAI adoption phases with quality evaluation practices and role responsibilities. This framework shows HOW quality is evaluated during GenAI adoption in phases and indicates WHO is accountable.

Fig. 5 consolidates our findings into a process-oriented view of GenAI adoption, with quality evaluations integrated into each phase and corresponding roles.

Each phase in the figure contains steps (white boxes), among which specific actions related to quality evaluation are highlighted. The ⋆ marks indicate points where evaluation is **directly enabled**, for example, assessing legal risks, verifying outputs, or analyzing system qualities. The ♦ marks identify **supporting checks**, for example, model prompting or installability that shape or constrain quality assurance decisions. Together, they form a network of quality signals – some proactive, others reactive.

Below each phase, the gray panels describe evaluation practices, which range from compliance vetting to metric-based evaluations. At the bottom, the roles responsible are connected, which reveals a distributed accountability structure, with legal, engineering, and quality teams all involved at different stages.

1) Quality evaluation practices in GenAI adoption: Based on collected data, three interconnected evaluations emerge: Gatekeeping – evaluation before development, Validation – testing during development, and Monitoring – evaluation during operation.

a) 1. ***Gatekeeping*** *(before development):* Evaluations in the Ideation phase are often conducted to determine whether a GenAI model or idea can be pursued. These gatekeeping checks are typically compliance-driven, involving legal and security teams.

The checklist in Table III was referenced by multiple participants as a tool to assess risk exposure and vendor accountability before prototyping. Participants from legal teams mentioned: *"We block use cases if the supplier can't commit to data privacy or copyright terms."* These evaluations serve as safeguards, especially in large organizations where regulatory pressure is high.

b) 2. ***Validation*** *(during development):* In the development phase, evaluation activities shift from permission to execution. Here, teams examine if an AI model behaves appropriately in their specific settings. This includes assessing data quality, response consistency, efficiency, and effectiveness with internal policies. We found that evaluation practices often lack mature tooling and rely on custom metrics. For example, a senior developer shared that *"We often create our specific key performance indicators in our implementation, aiming to give a signal on hallucinations."*

Table IV outlines a cross-functional practice that was used in multiple settings to structure quality assessments during implementation. These assessments are rarely isolated. They are tightly coupled to development iterations.

To better understand the relationship between the observed practices and software quality, we mapped the evaluation

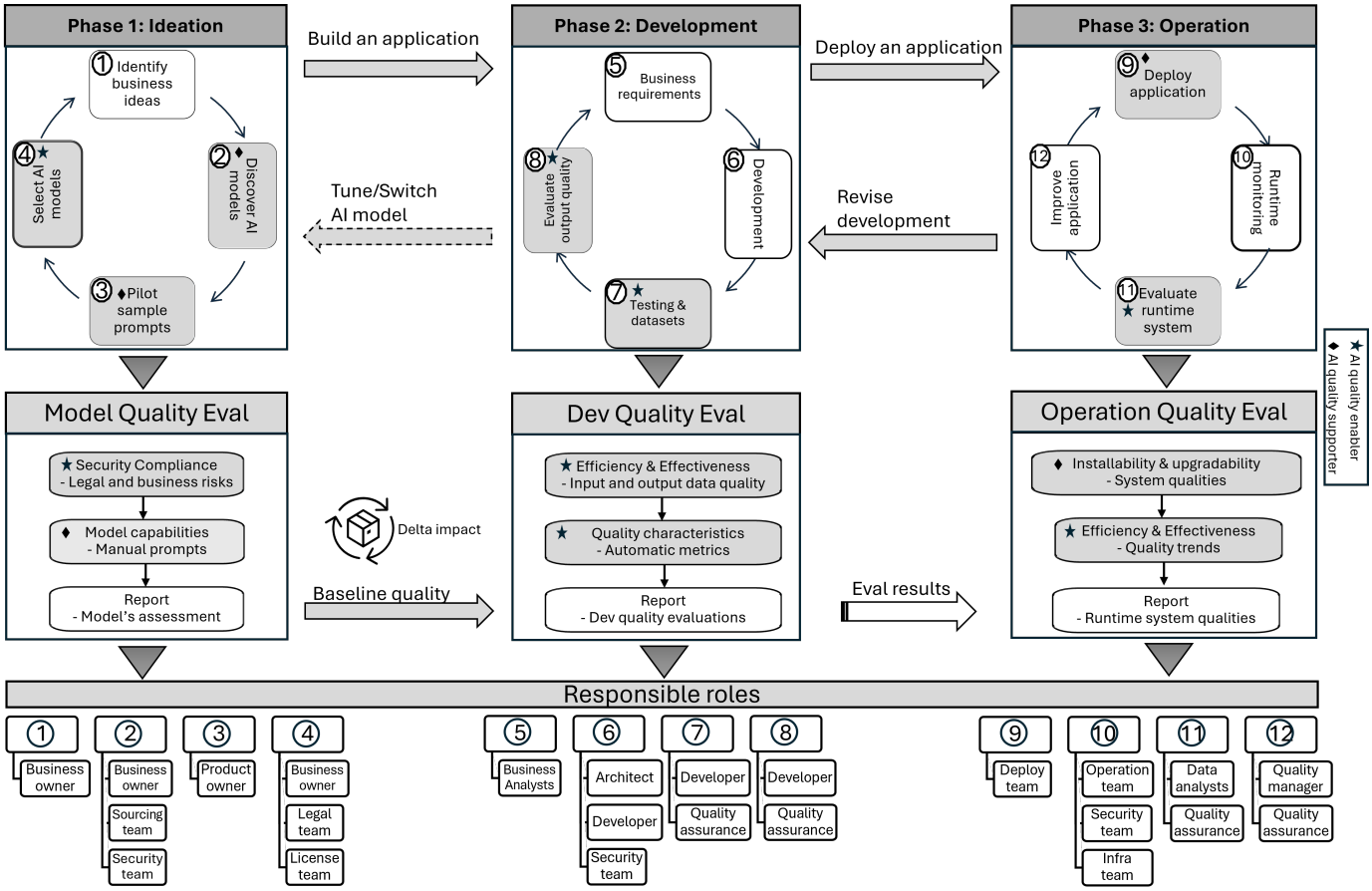

Fig. 5. Overview of quality evaluation framework during GenAI adoption in software development.

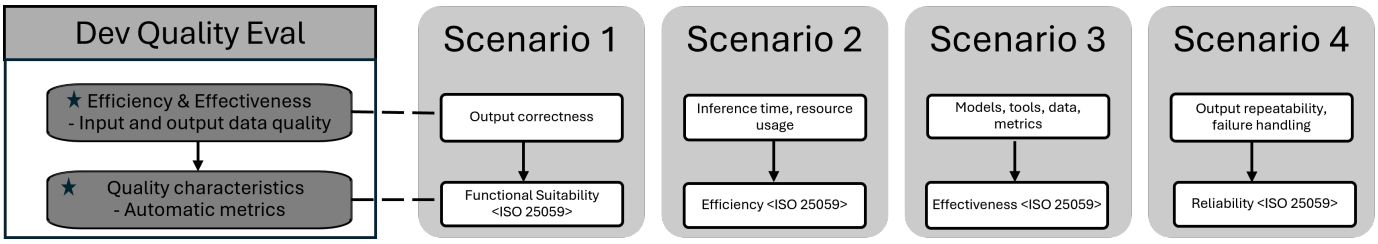

Fig. 6. Relation between observed Dev Quality Evaluation practices and ISO quality characteristics.

activities to ISO/IEC 25059 characteristics in Fig. 6.

As shown in Fig. 6, observed practices (e.g., prompt testing, the use of tools, and metrics) map implicitly to ISO quality characteristics. Specifically, they reflect:

- **Functional Suitability**. It includes the correctness and appropriateness of output.
- **Efficiency**. It covers measures for inference time duration and resource usage.
- **Effectiveness**. It covers task success and utility in the target context, assessed through targeted tests and usage evaluation.
- **Reliability**. It refers to output repeatability and failure handling.

These mappings indicate *what* to evaluate; they do not prescribe *how*. In our cases, metrics were selected per context, and some remained unsettled. Future tooling and policy definitions may benefit from translating these ISO sub-characteristics into practical checkpoints and observable metrics.

c) 3. ***Monitoring*** *(during operation):* Evaluation does not end at deployment. Once a system is deployed successfully, teams perform runtime assessments based on logs, live quality trends, and user feedback. This mode focuses on identifying performance degradation, systemic errors, or violations of usage policies. As shared by an operations engineer, *"It is required that we detect issues not when they happen, but when someone flags them downstream."* These practices are increasingly important in GenAI, where behavior can drift over time. Importantly, monitoring efforts feed back into development and legal reassessments, completing the lifecycle.

The integrated view in Fig. 5 reveals a decentralized architecture of quality assurance in GenAI adoption. Evaluation

TABLE III
LEGAL RISK ASSESSMENT CHECKLIST AND OWNERS

Legal risk type	Evaluation(s)	Responsible
Intellectual property infringement	Who is liable for infringement of copyrights in training data, e.g., the supplier or user?	Legal and Security teams
Training data	What data can be used to train/tune AI models?	Legal team
License	What Open Source Software has been used to develop the model(s), and which license applies?	Licensing team
Inputs of AI models	How do the AI suppliers treat users' confidential information, e.g., publicize user inputs or user inputs used to train AI models?	Legal and Security teams.
Outputs of AI models	Who is legally liable for the output of AI models? The supplier may disclaim all liabilities for use of their models	Legal team
Data privacy	Who is responsible for any illegal processing of personal data during the adoption of AI?	Legal and Security teams
	Can the AI model provider guarantee that the user data is not used to train the model, or in any other way processed by the provider of the system, or other third party for other purposes?	
	For any use cases of AI leading to recommendations, conclusions, or predictions related to individuals, can data protection for individuals be mitigated from the legal aspect?	

TABLE IV
QUALITY EVALUATIONS FOR GENAI APPLICATION IMPLEMENTATION

Development tasks	Evaluation(s)	Responsible
Information Security	How are objectives on confidentiality, integrity, and availability of information managed?	Security team
Data governance	How to handle data quality issues, unauthorized access, and improper use of data?	Security team
Contractual Frameworks	What are the terms of use for both internal AI adoption and for delivery towards the customer?	Legal team
Intellectual Property Rights	Avoid leakage of confidential information.	Licensing team
	No licensing out of patents.	
	Strict adherence to open source policies.	
Product Security	No use if no full control over AI tools/services.	Architect team
	A small and simple proof-of-concept is required.	
	AI is a support tool, not a decision maker.	
	Not be solely used for sensitive functionalities where incorrect outcomes cannot be tolerated.	
Data privacy	Complying with data protection principles/laws.	Legal team
	Processing user data only according to the customer contracts.	
	Maintaining records of processing use cases is a regulatory requirement.	
	No violations of individuals' rights and freedoms.	
Ethical and responsible outcomes	For in-house built models, training data should be explained and evaluated for bias.	Product owner Development team Quality team Management team
	For sourced models or tools, vendors must provide information about their bias analysis.	
	The ethical and responsible consequences of model error should be evaluated for the use case	
	Human-in-the-loop controls should be implemented.	
	Analyze all AI use cases as to their status against regulations. Ban those prohibited by law.	
	Classify AI use cases as limited- or high-risk under the AI Act.	
Quality of outcomes	PoC use cases to determine the seriousness of erroneous responses.	Product owner Development team Quality assurance team Management team
	Avoid use cases where the consequences are serious or no opportunity to check the response before action is taken.	
	Implement controls on the model training (data quality).	
	Inform the user about the possibility of error and the appropriate caution that should be taken.	
	Use application designs that allow source checking with source references.	
	Provide explainability tooling to help users understand how answers were determined.	
	Implement monitoring, response and support mechanisms for dealing with errors detected during production use.	

is not owned by a single role or function. It is distributed, contextual, and embedded in day-to-day decisions. Tables III and IV reflect how companies attempt to formalize what would otherwise remain tacit.

However, several challenges remain: metrics are inconsistent, practices vary across teams, and accountability is shared but unclear. Some participants described this as a lack of a coherent QA strategy tailored to GenAI, *"We have a security master in each dev team. But who is the quality champion for GenAI?"*

This observation motivates the discussion of a new role in Section VII-B, and underscores the practical value of treating quality evaluation not as an afterthought, but as an ongoing design activity.

2) Model reuse: The quality evaluation framework presented in Fig. 5 offers a reusable structure for organizations seeking to evaluate GenAI-enabled software.

By clarifying when and how evaluations are conducted, and by whom, the overview can serve as a reference for organizations lacking established quality practices for GenAI. It also helps bridge informal routines and ISO-inspired evaluation dimensions, as illustrated in Fig. 6. Therefore, the framework is a synthesized process reflecting actual engineering practice.

To further examine its practical utility, we conducted a case implementation in one of the studied companies. The aim was to apply the evaluation structure to a real GenAI adoption and assess its feasibility. The next section presents this framework verification case.

V. Verification of the Quality Evaluation Model

To verify the applicability and practicality of our quality evaluation framework, we implemented it in an internal software development project at one of the selected companies.

A. Case description

The case was a GenAI-assisted application for generating XML code used in Unstructured Supplementary Service Data (USSD) menus. USSD is a widely used communication protocol in mobile services that allows interaction through short codes without requiring internet access. The target users of this tool were merchants who needed to create dynamic menus for product browsing and purchasing without writing XML code manually. The tool enabled these users to provide natural language descriptions of their menu logic, which the system converted into valid, executable XML files for integration into mobile financial platforms.

This project was selected as a verification case for two reasons. First, it represents a realistic and commercially relevant GenAI adoption, involving code generation in a critical domain. Second, it required cross-role collaboration between developers, quality assurance engineers, legal reviewers, and product owners, reflecting the multi-stakeholder structure of GenAI adoption.

B. Application of the quality evaluation framework

The development process was structured around the phases and evaluation steps outlined in the quality evaluation framework.

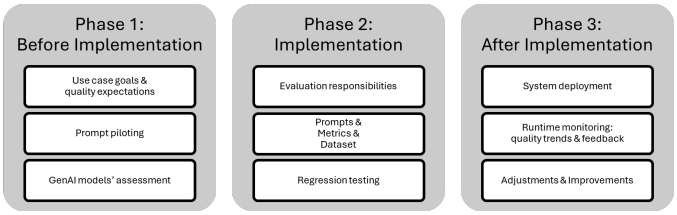

Fig. 7. The process of applying the quality evaluation framework

As shown in Fig. 7, in the **Before implementation** phase, the team specified use-case goals and quality expectations for GenAI-generated outputs, focusing on XML validity and semantic alignment. Prompt design and baseline testing were conducted with a suite of curated merchant inputs, and multiple GenAI models were assessed, including GPT-4o, Llama 3, and Gemini. Quality characteristics such as correctness, adaptability, and reliability were defined in relation to ISO/IEC 25059 and ISO/IEC 25023 guidance. The team selected GPT-4o based on output consistency and schema adherence in prompt testing. Legal and security experts reviewed the use case and verified that no sensitive or personal data would be handled, minimizing legal risk exposure.

In the **During implementation** phase, development work was integrated into quality evaluation steps. Specific responsibilities were assigned to developers, quality assurance (QA) engineers, and architects. Developers implemented prompt engineering improvements based on failure cases observed in a validation dataset. QA teams conducted regression testing and introduced a customized quality checklist inspired by the tables proposed in Section IV and III.

Evaluation covered functional suitability, efficiency, and correctness, using metrics such as XML validity rate, response time, and coverage of logic branches. This aligns with the middle phase of the framework, where evaluation activities are embedded in GenAI application tasks.

In the **After implementation** phase, the system was deployed to a live production environment with monitored usage. Grafana dashboards [28] were used to visualize runtime quality indicators. Metric scores, such as output validity and latency, were collected using Prometheus Pushgateway and logged over time. Feedback from merchant users was also monitored to detect misunderstood or misrendered menu structures. Operational adjustments were made based on findings, including prompt template refinement and rule-based XML post-validation. This phase corresponded to the framework's final stage and enabled long-term quality tracking and post-deployment tuning.

C. Verification outcomes

The proposed framework was applied to all three phases of the USSD XML Generator case. It enabled the team to address quality concerns with defined responsibilities, metrics, and checkpoints.

In the early phase, the model assessment helped filter out unsuitable options and anticipate risks. Legal and security engineers used the checklist to validate compliance concerns before deployment. One engineer noted, "The checklist turned vague worries into actionable points, such as licensing, datasets, inputs, and outputs." During development, quality metrics were embedded into the implementation process. Accuracy and latency were measured continuously using in-house scripts and Prometheus monitoring [29]. A quality assurance engineer remarked, "The quality characteristics gave us criteria early, so we knew what to test and why." Operational monitoring added a feedback loop post-deployment. Monitoring dashboards showed runtime trends that directly informed model prompt adjustments. "Seeing the correctness rate of generated outputs improve week by week gave us real confidence," said an operations engineer.

Despite its practical value, the framework had limitations during the case implementation. First, the success of its application relied on the willingness and capacity of team members to take ownership of additional evaluation tasks. Second, although the framework included evaluation practices with ISO alignment, the evaluation results still depended on the quality of test data and prompt engineering, where guidance was limited. Third, the monitoring approach, while useful, required specific tooling and metrics setup that may not generalize to other organizations.

We position the framework as a meta-level structure with a stable core (checkpoints, role duties, and a minimal metric set) and configurable parts (attributes, datasets, and safeguards). Teams can instantiate it per feature or domain using shared

templates and steps, rather than building a new method or bringing new tools each time. In the USSD case, the core metrics were reused; prompts and test data were tailored.

VI. Validity Threats

We follow Runeson et al.'s guideline to address validity threats: construct, internal, external, and conclusion validity [30].

a) Construct validity: To identify quality evaluation practices in GenAI adoption, we conducted 19 interviews with practitioners who had hands-on experience with GenAI-supported software development. Participants were selected from multiple roles, including development, testing, architecture, legal, and management, to ensure diverse perspectives. Although each participant had experience with parts of the entire process, their combined accounts provided a holistic view across its constituent steps. However, participants may have used inconsistent terminology or lacked shared definitions of "quality," "evaluation," or "GenAI adoption." To mitigate this, we clarified questions during interviews (e.g., "What did your team do to ensure the model was suitable?"), followed by clarifying questions and validation of summaries during transcription checks.

b) Internal validity: Our thematic analysis followed a staged coding process, including initial open coding, coding for pattern identification, and iterative refinement. Multiple researchers reviewed the coding structure and figure derivation to minimize personal bias. However, participant statements may still include misinterpretations. We mitigated this through three group sessions to review collected artifacts (e.g., internal documents and metrics). For the verification case, the same research team collaborated with practitioners during implementation, which may introduce bias toward confirming our proposed framework. To reduce this, evaluation metrics and improvement steps were interpreted by study participants.

c) External validity: In this study, we selected two software development companies from the same corporate group, which may introduce shared culture and governance effects. The framework was derived in a setting with development teams, in-house legal and security support, and cross-site collaboration. Organizations that lack these prerequisites, such as smaller teams without dedicated risk or operations functions, may need to adapt roles, checkpoints, and metrics before adoption. Therefore, we present the framework as process-oriented guidance rather than a universal prescription, and we distinguish elements from configurable ones to aid tailoring. The verification case demonstrates feasibility in the studied context, but applying the framework elsewhere will require adaptations to local constraints.

d) Conclusion validity: We documented research procedures, coding steps, and analytical decisions to support transparency. The first three researchers analyzed the data and resolved differences to ensure reliability. Participants' feedback was used to validate our interpretations of key themes.

We acknowledge potential threats to conclusion validity. For example, if the data concerning legal evaluation practices were incomplete or misinterpreted, an alternative conclusion might be that legal responsibilities are decentralized rather than coordinated through a central unit. However, our collected data showed that legal evaluation was explicitly addressed by multiple participants. Furthermore, our multi-step verification, such as peer checking, participant feedback, and triangulation, reduces the risk of such misinterpretations.

Additionally, we reported both positive outcomes and challenges during the verification of the proposed framework to mitigate confirmation bias. Future studies are needed to understand the framework's long-term effects in other industrial settings.

VII. Discussions

A. Legal risk and security compliance come first?

In our cases, preliminary legal and security checkpoints blocked or delayed pilots. The reasons included supplier terms that allowed training on user inputs, unclear ownership of generated code, and data-residency constraints that conflicted with company policy (Table III). While many engineering activities are iterative and exploratory, legal and compliance violations can trigger irreversible consequences, such as breach of contract, regulatory sanctions, or reputational damage. This risk profile justifies the placement of legal and security evaluation as the first quality gates in our framework.

The necessity of prioritizing these concerns became evident in our interviews. As one participant noted: "We did not even get to try the model, legal shut it down because of the terms of service." Unlike performance issues or usability flaws, legal violations typically cannot be patched silently once a product reaches customers. This means the cost of deferred evaluation is not technical debt but organizational risk. Treating these issues as upstream concerns helps ensure that software built on GenAI is legally viable and security-resilient before other quality aspects are even meaningful.

Additionally, the placement of these assessments early in the workflow promotes shared accountability across teams. Rather than delegating compliance checks to external gatekeepers, integrating them into quality evaluation encourages engineers to consider these aspects as part of their technical responsibility. This shift from reactive compliance to proactive governance requires extra resources to deal with the risk and compliance evaluation.

B. Introducing a GenAI quality lead role

Our findings show that quality evaluation responsibilities in GenAI-supported software development are distributed across several functional roles, including legal teams, developers, security engineers, analysts, and quality managers. While each role addresses a specific concern—such as compliance, technical performance, or operational monitoring—no designated function oversees quality holistically across the entire lifecycle.

This absence of centralized responsibility leads to fragmentation in evaluation practices and limited traceability of decisions. Practices such as legal risk assessment or runtime evaluation are conducted in isolation, without a unifying

perspective that ensures GenAI-specific quality concerns are systematically addressed.

To mitigate this gap, we propose the establishment of a dedicated role: the *GenAI Quality Lead (GQL)*. The GQL is envisioned as a cross-functional role that ensures quality considerations are proactively embedded throughout the GenAI adoption lifecycle—from model selection and data preparation to deployment and continuous operation.

Specifically, the GenAI Quality Lead would coordinate the following responsibilities:

- Collaborate with legal and security teams during model sourcing to address compliance and licensing risks.
- Work with developers and analysts to define evaluation criteria aligned with business and technical requirements.
- Ensure structured assessment of output quality and runtime behavior, drawing on relevant metrics and feedback loops.
- Facilitate alignment with external standards, such as ISO/IEC 25059, by translating quality characteristics into actionable practices.

This role draws conceptual support from the SE4AI (Software Engineering for AI) perspective, which emphasizes the need for dedicated quality assurance practices tailored to AI-based systems. The GenAI Quality Lead does not replace existing domain experts, but rather functions as a coordinating actor that integrates diverse quality concerns into a consistent and traceable process.

Establishing such a role may improve organizational capability to systematically evaluate GenAI applications and adapt to emerging regulatory and standardization landscapes.

C. Why ISO quality characteristics matter

Across the organizations we studied, quality was interpreted through different aspects: efficiency by developers, explainability by managers, and compliance by legal teams. This fragmentation created challenges in aligning expectations, coordinating evaluations, and making trade-offs explicit. What was missing was a common foundation to reason about quality without flattening its complexity.

ISO/IEC 25059 fills this gap by offering a standardized structure of quality characteristics tailored to AI-enabled systems. It may be more recognizable to software engineering professionals compared to custom system qualities. In our implementation, ISO characteristics helped participants translate vague concerns into actionable targets. For instance, instead of "the model should be fast and safe," teams articulated expectations as "inference time < 45 seconds (efficiency)" and "no confidential leakage from enterprise information (security)." This shift enabled traceable decisions and role-specific accountability throughout the lifecycle.

Moreover, ISO's separation of characteristics and sub-characteristics (e.g., from reliability to robustness) allowed practitioners to tailor depth without discarding structure. This flexibility appears important in real-world development, where not all quality attributes are equally relevant. The standard helps teams organize what they have already done, while exposing what might be missed in terms of software quality.

We acknowledge that ISO 25059 is still under refinement and does not address all GenAI-specific characteristics. However, our evidence suggests that it provides a shared model grounded in both academic rigor and industrial recognition.

a) Relation to the EU AI Act: While ISO/IEC quality standards help structure technical quality, they do not by themselves address regulatory duties under the EU AI Act (e.g., risk classification, documentation, conformity assessment, post-market monitoring). When this study started, the operational guidance for the Act was still evolving in the studied companies, and the usage of software development was considered unclear. Therefore, teams mapped existing ISO-based quality characteristics to internal policies for GenAI adoption and usage, conducted legal and privacy reviews, and prepared for later alignment with evolving standards and regulatory requirements, not limited to ISO/IEC 25059.

Future studies should investigate how GenAI quality evolves over time and how teams update evaluation criteria accordingly. Moreover, the GenAI Quality Lead role needs deeper exploration, including required competencies and interactions with existing quality assurance activities.

VIII. Conclusions

The use of GenAI technologies in industrial software development introduces new challenges in software quality assurance. Through a case study in two industrial organizations, we investigated the process of GenAI adoption in practice, what quality aspects matter, who is responsible for evaluation, and how these activities are carried out across the software development.

In the study, we contribute a framework: a process-oriented and role-aware view of GenAI quality evaluation. The framework is built based on observed practices and practitioners' feedback. This framework clarifies when and how quality should be assessed between legal, technical, and operational responsibilities.

By visualizing where evaluation takes place and what characteristics are considered, we help practitioners gain a deeper understanding of GenAI-related quality. The introduction of the GenAI Quality Lead role addresses a missing coordination function, supporting traceability, consistency, and compliance in multi-role environments. The real-world implementation further confirms the framework's applicability and exposes opportunities for refinement.

Looking forward, GenAI adoption can accelerate across sectors and development contexts. It raises the necessity for quality evaluation frameworks that are not only theoretically grounded but also implementable in practice. Our study offers such a framework for practitioners to follow, and also provides a lens to rethink software quality in the era of GenAI.

Acknowledgement

We acknowledge support from the KKS Foundation through the S.E.R.T. Research Profile Project at Blekinge Institute of Technology.

DATA AVAILABILITY

The datasets supporting the conclusions of this article are available in the IEEEDataPoint [25].

REFERENCES

[1] S. Feuerriegel, J. Hartmann, C. Janiesch, and P. Zschech, "Generative ai," *Business & Information Systems Engineering*, vol. 66, no. 1, pp. 111–126, 2024.

[2] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, and Y. Wang, "Towards an understanding of large language models in software engineering tasks," *Empirical Software Engineering*, vol. 30, no. 2, p. 50, 2025.

[3] J. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine learning testing: Survey, landscapes and horizons," *IEEE Transactions on Software Engineering*, vol. 46, no. 10, pp. 1070–1093, 2020.

[4] L. Yu, "Paradigm shift on coding productivity using genai," *arXiv preprint arXiv:2504.18404*, 2025.

[5] P. d. O. Santos, A. C. Figueiredo, P. Nuno Moura, B. Diirr, A. C. Alvim, and R. P. D. Santos, "Impacts of the usage of generative artificial intelligence on software development process," in *Proceedings of the 20th Brazilian Symposium on Information Systems*, 2024, pp. 1–9.

[6] A. Aleti, "Software testing of generative ai systems: Challenges and opportunities," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 4–14.

[7] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in Neural Information Processing Systems*, vol. 28, pp. 2503–2511, 2015.

[8] C. T. Ungureanu and A. E. Amironesei, "Legal issues concerning generative ai technologies," *Eastern Journal of European Studies*, vol. 45, 2023.

[9] V. Riccio, G. Jahangirova, A. Stocco, N. Humbatova, M. Weiss, and P. Tonella, "Testing machine learning based systems: A systematic mapping," in *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '20. ACM, 2020, pp. 1–12.

[10] J. Park and S. Choo, "Generative ai prompt engineering for educators: Practical strategies," *Journal of Special Education Technology*, p. 01626434241298954, 2024.

[11] A. Donvir, S. Panyam, G. Paliwal, and P. Gujar, "The role of generative ai tools in application development: A comprehensive review of current technologies and practices," in *2024 International Conference on Engineering Management of Communication and Technology (EMCTECH)*. IEEE, 2024, pp. 1–9.

[12] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical software engineering*, vol. 14, no. 2, p. 131, 2009.

[13] J. Wang and Y. Chen, "A review on code generation with llms: Application and evaluation," in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 2023, pp. 284–289.

[14] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "Using deep learning to generate complete log statements," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. ACM, 2022, pp. 883–895.

[15] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pre-trained language models for text generation: A survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–39, 2024.

[16] M. Simaremare and H. Edison, "The state of generative ai adoption from software practitioners' perspective: An empirical study," in *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2024, pp. 106–113.

[17] International Organization for Standardization, "Iso/iec 25059:2023 software engineering — systems and software quality requirements and evaluation (square) — quality model for ai systems," *International Organization for Standardization*, 2023.

[18] ——, "Iso/iec 25010:2011 systems and software engineering — systems and software quality requirements and evaluation (square) — system and software quality models," *International Organization for Standardization*, 2011.

[19] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ml test score: A rubric for ml production readiness and technical debt reduction," *IEEE Big Data*, pp. 1123–1132, 2017.

[20] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of nlp models with checklist," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4902–4912.

[21] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," in *Proceedings of the 44th ACM/IEEE International Conference on Software Engineering*, ser. ICSE '22. IEEE, 2022, pp. 319–331.

[22] U.-e. Habiba, M. Haug, J. Bogner, and S. Wagner, "How mature is requirements engineering for ai-based systems? a systematic mapping study on practices, challenges, and future research directions," *Requirements Engineering*, vol. 29, no. 4, pp. 567–600, 2024.

[23] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén *et al.*, *Experimentation in software engineering*. Springer, 2012, vol. 236.

[24] K. Petersen and C. Wohlin, "Context in industrial software engineering research," in *2009 3rd International Symposium on Empirical Software Engineering and Measurement*. IEEE, 2009, pp. 401–404.

[25] L. Yu, "A process model for evaluating genai adoption and use in software development," 2025. [Online]. Available: https://dx.doi.org/10.21227/zhbk-fc92

[26] D. S. Cruzes and T. Dyba, "Recommended steps for thematic synthesis in software engineering," in *2011 International Symposium on Empirical Software Engineering and Measurement*. IEEE, 2011, pp. 275–284.

[27] K. Kenthapadi, H. Lakkaraju, and N. Rajani, "Generative ai meets responsible ai: Practical challenges and opportunities," in *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, 2023, pp. 5805–5806.

[28] Grafana Labs, "Grafana: Open-source visualization and dashboards," https://grafana.com/grafana, 2025, accessed 2025-08-01.

[29] Prometheus, "Prometheus: Monitoring for your systems and services," https://prometheus.io, 2025, accessed 2025-08-01.

[30] P. Runeson, M. Höst, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*. Hoboken, NJ: Wiley, 2012.